

パチスロ設定推測アプリ

詳細設計書

対象プロジェクト: SlotSettingAnalyzerCompleteV2
基準時点: 2026-03-23 / 現在の実装（セッション履歴・再開フロー反映版）

アプリ目的	実戦中の入力値から設定推測を行い、推測結果を履歴として保持し、後から再確認・再開できるようにする。
技術構成	Android / Kotlin / Jetpack Compose / Clean Architecture + MVVM / Room / Hilt / Navigation Compose
現在の主機能	機種選択、実戦入力、推測結果、実戦終了、履歴一覧、履歴詳細、履歴からの再開
機種マスタ	assets JSON + Seeder 方式。manifest 管理あり。登録済み 4 機種（sample_machine_a / hokuto / monster_hunter / kabaneri）

1. ドキュメントの目的と前提

1.1 目的

本書は、現行 Android アプリの実装内容をベースに、画面仕様、データ構造、画面遷移、推測・履歴フロー、Git 反映手順までを 1 冊にまとめた詳細設計書である。前回の V3 設計書を土台としつつ、実装済みのセッション ID ベス遷移、履連携、推測結果スナップショット保存、履からの再開機能を反映している。

1.2 スcope

象は app モジュールの現行構成一式であり、特に Home / MachineSelect / SessionInput / Inference / HistoryList / HistoryDetail の 6 画面、Room テーブル、Seeder、UseCase、Repository、および Git 反映用の運用メモを含む。

1.3 設計上の考え方

本アプリは“今打っている台の設定推測”を最優先とし、台データの長期統計よりも、片手操作、短時間入力、推測確認、後からの見返し・再開を重視した構成とする。

2. システム概要

2.1 利用シナリオ

ユザはホームから機種を選び、新規を開始する。入力面で回や BIG/REG/CZ/AT 等を更新し、推測結果面で各設定スコアと設定スコアを確認する。終了時にはセッションを完了態にし、履歴一覧から見返したり、履歴詳細からセッションを再開できる。

2.2 現在の機能範

実装済み: 機種マスタ投入、機種検索、セッション作成/再利用、動的カウンタ入力、確率表示、推測スコア表示、信頼度表示、推測理由表示、実戦終了、履歴一覧、履歴詳細、履歴からの再開。未実装寄り: 広告、クラウド同期、iOS 展開、自動機種追加管理。

2.3 現在の登録機種

manifest には sample_machine_a、hokuto、monster_hunter、kabaneri の 4 機種が定義されている。Seeder は masterVersion を比較し、差分がある場合のみ再投入する。

3. アーキテクチャ設計

3.1 採用アーキテクチャ

Presentation / Domain / Data の 3 層を基本とした Clean Architecture + MVVM を採用する。UI は Compose、状態は ViewModel の UiState、永続化は Room、DI は Hilt、画面遷移は Navigation Compose とする。

3.2 依存関係

Presentation は UseCase を経由して Domain に依存し、Repository Interface の実体は Data 層が提供する。画面から DAO や JSON へ直接アクセスしない。

3.3 上の特

セッション管理は sessionId を基準とする。これにより“現在の実戦”だけでなく、履歴から選んだ過去セッションの再表示や再開が可能になっている。

4. プロジェクト構成

現行ソースは feature 単位で整理されており、core / data / domain / di / ui に分離されている。主要パスを以下に示す。

パス	役割
core/navigation	AppRoutes と AppNavHost を保持。sessionId を引に各面を遷移させる。
core/ui	ScoreBar コンポーネントなど共通 UI を保持。
data/database	Room DB, DAO, Entity, Relation を保持。

data/datasource	assets 読み込み、manifest 解析、バリデーション、Seeder を保持。
data/repository	Repository 実装。Machine / Session / History などの永続化入口。
domain	Model / Repository Interface / UseCase を保持。
feature/home	ホーム画面。開始導線と履歴導線。
feature/machine	機種選択、検索、セッション開始/再開。
feature/session	実戦入力。数値更新、初期化、推測画面への遷移。
feature/inference	推測ロジック実行結果の表示、実戦終了。
feature/history	履歴一覧と履歴詳細、再開導線。
di	DatabaseModule / RepositoryModule / SeederModule を保持。

5. 画面遷移設計

AppRoutes では HOME / MACHINE_SELECT / SESSION_INPUT/{sessionId} / INFERENCE/{sessionId} / HISTORY_LIST / HISTORY_DETAIL/{sessionId} を定義している。sessionId をルートに埋め込むことで、対象セッションを明示的に扱う構成にしている。

画面	ルート	主な遷移元	主な遷移先
Home	home	アプリ起動	MachineSelect / HistoryList
MachineSelect	machine_select	Home	SessionInput(sessionId)
SessionInput	session_input/{sessionId}	MachineSelect / HistoryDetail	Inference(sessionId)
Inference	inference/{sessionId}	SessionInput	HistoryList / るで SessionInput
HistoryList	history_list	Home / Inference	HistoryDetail(sessionId)
HistoryDetail	history_detail/{sessionId}	HistoryList	SessionInput(sessionId) 再開

6. 画面仕様

6.1 Home

役割	新規実戦開始と履歴閲覧への入口。登録機種数も表示する。
主 UI	『新しく実戦を始める』 『履歴を見る』の導線ボ

	タン。
備考	登録機種数を <code>state.machines.size</code> から表示する。

6.2 MachineSelect

役割	機種検索と、機種単位の新規実戦開始 / 続きから再開。
主 UI	検索欄、機種一覧、選択中表示、新規実戦開始ボタン、続きから再開ボタン。
理	<code>observeMachinesUseCase</code> で有効機種一覧を監視。 <code>startNewSession</code> は強制新規、 <code>resumeSession</code> は既存未終了セッションがあれば再利用。

6.3 SessionInput

役割	実戦中の数値をその場で入力するメイン画面。
主 UI	機種名表示、説明カード、機種ごとのカウンタ入力欄、入力値初期化ボタン、推測結果へボタン。
理	表示対象は機種マスタの <code>isEnabled</code> / <code>isDefaultVisible</code> 等で制御。 <code>total_games</code> は 1/10/100/1000、それ以外は 1/10/100 ステップ。値更新時に Room へ即時反映。

6.4 Inference

役割	設定推測結果の確認と実戦終了。
主 UI	入力値一覧、推測理由、最有力設定、設定 1～6 バー、低/中/高設定帯バー、信頼度、サマリー、実戦終了ボタン。
理	<code>loadInference</code> 時に <code>CalculateInferenceUseCase</code> を実行し、結果要約と <code>confidence</code> をセッションへスナップショット保存。 <code>finishSession</code> 実行で <code>isFinished</code> と <code>endedAt</code> を更新。

6.5 HistoryList

役割	終了済みセッション中心の履歴一覧表示。
主 UI	機種名、日付、総ゲーム数、サマリー、信頼度、状態ラベル。

処理	observeFinishedSessionsUseCase の結果を画面表示用 UI モデルに変換し、未計算時は『推測結果を確認してください』を表示。
----	--

6.6 HistoryDetail

役割	履歴の推測結果再計算表示と再開導線。
主 UI	機種名、実戦日、総ゲーム数、BIG/REG 確率、CZ/AT 回数、設定バー、設定帯バー、サマリー、再開ボタン。
処理	sessionId を受けてセッションと機種を読み込み、CalculateInferenceUseCase で再度結果を算出。再開時は reopenPlaySessionUseCase で current 化して SessionInput に戻す。

7. データベース設計

Room にはマスタ系、セッション系、履歴系、設定系のテーブルが存在する。現行実装で重要な主テーブルを以下に整理する。

テーブル	用途	主キー/主項目	補足
machines	機種マスタ	id, name, isActive	Seeder で投入
machine_counter_definitions	入力項目定義	machine_id, key	表示名、単位、sortOrder、isEnabled 等を保持
machine_setting_reference_values	理論値	machine_id, counter_key, setting_no	1/x の分母理論値等を保持
machine_ceiling_rules	天井ルール	machine_id, counter_key 等	将来の天井表示用
play_sessions	実戦セッション	id, machine_id, started_at, updated_at, ended_at, is_finished, is_current	推測スナップショット summary / confidence も保持
session_counter_values	実戦カウンタ値	session_id, counter_key, int_value	回や BIG 等を保持
play_histories	履歴スナップショット	history_id 等	現構成ではセッション履歴中心だがテーブルは残置
app_settings	ユーザー設定	key, value	並び順等の設定用

master_metadata	マスタ管理	master_version, updated_at	Seeder の投入判定に使用
-----------------	-------	-------------------------------	-----------------

7.1 play_sessions 主要項目

現行フローの中心になるため、play_sessions の主要カラムを抜粋する。

項目	意味	備考
id	セッション ID (UUID)	画面遷移のキー
machine_id	対象機種 ID	マスタ参照用
machine_name_snapshot	当時の機種名	履歴表示のため保持
started_at / updated_at / ended_at	開始・更新・終了時刻	履歴表示と状態判定に使用
is_finished	終了済みか	Inference で終了処理後に true
is_current	現在セッションか	再開時に付け替える
is_archived_in_history	履歴アーカイブ済みフラグ	将来拡張余地あり
last_inference_summary	最後の要約文	履歴一覧へ表示
last_confidence_label	最後の信頼度	HIGH/MEDIUM/LOW 等を保持

8. ドメイン・UseCase 設計

主要ドメインモデルは Machine、MachineCounterDefinition、SettingReferenceValue、PlaySession、SessionCounterValue、InferenceResult、SettingScore、UserPreferences である。

UseCase	役割	主利用画面
ObserveMachinesUseCase	有効機種一覧の監視	MachineSelect
StartPlaySessionUseCase	新規開始 or 再開セッション取得	MachineSelect
GetSessionUseCase	セッション読み込み	SessionInput / Inference / HistoryDetail
UpdateCounterUseCase	カウンタ更新	SessionInput
ResetCurrentSessionCountersUseCase	全カウンタ 0 初期化	SessionInput
CalculateInferenceUseCase	設定スコア算出	Inference / HistoryDetail
UpdateSessionInferenceSnapshotUseCase	summary と confidence を保存	Inference
FinishPlaySessionUseCase	実戦終了処理	Inference
ObserveFinishedSessionsUseCase	履歴一覧監視	HistoryList

ReopenPlaySessionUseCase	履歴セッションを再開	HistoryDetail
GetMachineUseCase	機種マスタ読み込み	複数画面

9. 推測ロジック設計

推測ロジックは機種ごとの理論値（settingReferenceValues）と、セッションの実測値を比較して設定スコアを正規化したうえで表示する。UI では設定 1～6 の個別バーと、低設定帯(1-2) / 中間設定帯(3-4) / 高設定帯(5-6) の帯別バーを併用する。

観点	現行実装の内容
入力値一覧	回は「 n G」、その他は「 回 / 確率 1/x」で表示
サンプル数評価	5000G 以上=強い、3000G 以上=ある程度判断可、1000G 以上=暫定、未満=サンプル不足
理由表示	各カウンタの実測 1/x と最も近い理論値の設定を比較し、どの設定に近い文章化
最有力設定	normalizedValue が最大の設定を採用し、パーセンテージ表示
信頼度表示	confidenceLabel を「高 / 中 / 低 / サンプル不足」等に変換
履歴反映	推測実行時に lastInferenceSummary と lastConfidenceLabel をセッションへ保存

10. セッション・履歴フロー

セッションの開始から履歴再開までの流れを以下にまとめる。

段階	主処理	更新対象	結果
新規開始	startSession(machineId)	play_sessions, session_counter_values	新しい UUID セッションを作成し、全カウンタ 0 で初期化
続きから再開	getOrCreateSessionForMachine(machineId)	play_sessions.is_current	未終了セッションがあれば current 化して再利用
入力更新	updateCounter(command)	session_counter_values, play_sessions.updated_at	値変更を即時永続化
推測確認	CalculateInferenceUseCase + snapshot 保存	play_sessions.last_inference_*	履歴一覧に要約と信頼度が出る

実戦終了	finishSession(sessionId)	ended_at, is_finished	履歴一覧に載る状態へ遷移
履歴表示	observeFinishedSessions	画面 UI state	終了日・状態・サマリーを表示
履歴から再開	reopenSession(sessionId)	is_current, is_finished, updated_at	SessionInput に戻り続き入力が可能

11. 機種マスタ投入設計

Seeder は machine_master_manifest.json を読み込み、masterVersion を比較して差分投入を行う。手順は manifest 読み込み → machine JSON 読み込み → parser → validator → importer → version 保存。バリデーション失敗または import 例外時は SeedResult にエラーを詰めて中断する。

この構成により、アプリ更新時に assets の JSON を差し替えるだけでマスタ更新できる。将来クラウド同期や外部 CMS に移行する場合も、Source 層の差し替えで対応しやすい。

12. 現在の到達点と改定リスト

区分	反映内容	態
画面遷移	sessionId 付きの SessionInput / Inference / HistoryDetail 遷移へ移行	実装済み
SessionInput	機種別動的入力、確率表示、直接入力、初期化ダイアログ	実装済み
Inference	設定バー / 設定帯バー / 理由文 / 推測スナップショット保存 / 実戦終了	実装済み
履歴一覧	終了済みセッション監視、サマリー・信頼度・状態表示	実装済み
履歴詳細	再計算表示、BIG/REG 確率、CZ/AT 回数、再開ボタン	実装済み
機種マスタ	4 機種 assets 反映、manifest バージョン管理	実装済み
広告・クラウド同期	AdMob 本格導入、クラウド経由マスタ更新	未着手/今後

13. Git 反映方針

今回の成果物は、設計書更新ファイル・改定リスト・Git 手順メモを docs/ 配下へ追加してコミットする想定とする。ソース改修が含まれる場合は同時に add してよいが、設計書だけを先に反映したい場合は docs だけを分けてコミットすると履が追やすい。

- 1) 作業ブランチを切る: `git checkout -b docs/v0.4-design-refresh`
- 2) 追加ファイルを配置: `docs/SlotSettingAnalyzer_詳細設計書_v0_4.docx`,
`docs/CHANGELOG_v0.4.md`, `docs/GIT_PUSH_GUIDE.md`
- 3) ステージング: `git add docs/`
- 4) コミット: `git commit -m "docs: refresh design spec and revision list"`
- 5) 反映: `git push -u origin docs/v0.4-design-refresh`
- 6) main に載せる場合は GitHub またはローカルでマージ後、`git push origin main`

14. 補足と今後の候補

今後は、広告導入、マスタ配信の外部化、設定推測ロジックの重み付け強化、機種別専用入力 UI、履歴の絞り込み/タグ付け、CSV 出力などが張候補になる。現時点では “現場で素早く入力して推測確認する” という主目的に対して、十分に一本筋の通った構成になっている。